

# IntelliSudoku

## *Comparative Analysis of AI Search Algorithms for Sudoku Solving*

### Team Members

- Kazi Dider Sayed — 0112330104
- Mosfiqur Rahman — 0112331034
- KM Labib — 0112330240

### Problem Statement

Sudoku is a constraint-based puzzle that can require exploring a large number of possible solutions, especially for difficult puzzles. This project investigates how different AI search algorithms and heuristics can improve the efficiency of Sudoku solving. Multiple approaches, including basic backtracking, Minimum Remaining Values (MRV), Forward Checking, and constraint propagation (AC-3), will be implemented and compared based on solving time, search nodes explored, and number of backtracks.

### Motivation

Sudoku provides a simple but challenging environment for studying Artificial Intelligence search techniques. Different algorithms can solve the same puzzle with very different levels of efficiency. By comparing these approaches, this project aims to understand how AI heuristics and constraint-propagation techniques reduce unnecessary search and improve solving performance, especially for the hardest, most under-constrained puzzles (e.g., minimal 17-clue instances).

### Dataset

The project will use a collection of Sudoku puzzles with different difficulty levels, including Easy, Medium, Hard, and Expert. The dataset will contain approximately 400–1,000 puzzles, allowing the AI algorithms to be tested under different levels of complexity.

### Source and Access Method

- Primary source: a publicly available, pre-verified Sudoku puzzle dataset (e.g., the Kaggle "1 Million Sudoku Games" dataset or the public 17-clue minimal-puzzle collection), downloaded directly as CSV files under an open license.
- Supplementary source: a small set of puzzles generated programmatically using a constraint-based generator-and-solver script, to guarantee coverage across all four difficulty bands and to include a subset of known minimal (17-clue) puzzles for stress-testing.
- Difficulty labeling: puzzles are bucketed into Easy/Medium/Hard/Expert using the number of initially filled cells and, where available, the difficulty rating provided by the source dataset.

Each puzzle record will contain: puzzle grid, difficulty level, known/verified solution, and number of initially empty cells. The same set of puzzles will be given to all algorithms to ensure a fair and consistent comparison.

### Proposed Methodology

The project will implement and compare multiple AI search algorithms for solving Sudoku puzzles. First, the puzzles will be preprocessed and categorized by difficulty level. Then, the following solving approaches will be applied:

- Basic Backtracking — used as the baseline algorithm.
- Backtracking + MRV — selects the cell with the fewest possible values.
- Backtracking + Forward Checking — checks future possibilities to detect invalid paths early.
- MRV + Forward Checking — combines both heuristics for improved search efficiency.
- AC-3 Constraint Propagation (extended baseline) — applies arc-consistency to prune the domain before and during search, added to give the comparison a non-trivial constraint-satisfaction technique beyond simple heuristic ordering.

All algorithms will solve the same set of puzzles, and their performance will be recorded and compared.

**General Framework:** *Sudoku Dataset* → *Preprocessing* → *AI Algorithms* → *Performance Measurement* → *Comparison & Analysis*

### Evaluation Plan

The AI algorithms will be evaluated by solving the same set of Sudoku puzzles across different difficulty levels. Basic Backtracking will be used as the baseline for comparison. Performance will be compared using the following metrics:

- Solving Time — time required to solve each puzzle.
- Search Nodes Explored — number of possible states examined during solving.
- Number of Backtracks — number of times the algorithm needs to backtrack.
- Success Rate — percentage of puzzles successfully solved within a fixed time/node budget (to handle worst-case blow-up on Expert puzzles).

Results will be presented using tables and graphs (e.g., solving time vs. difficulty, nodes explored per algorithm) to determine which AI approach provides the best performance, particularly for harder puzzles.

## Proposed System

The proposed system will be a Sudoku-solving platform that allows users to select a Sudoku puzzle and apply different AI algorithms to solve it. The system will provide a comparison of the algorithms based on their performance. The system will include:

- Sudoku puzzle input
- Multiple AI solvers (Basic Backtracking, Backtracking + MRV, Forward Checking, MRV + Forward Checking, AC-3)
- Performance measurement
- Algorithm comparison
- Solving statistics and results

**Proposed Architecture:***Sudoku Puzzle → AI Solver → Solution → Performance Analysis → Comparison*

## Timeline / Milestones

| Week   | Milestone                                                                                                                                                             | Deliverable                                                             |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Week 1 | Literature review; finalize dataset source; acquire/generate Sudoku puzzle dataset (400–1,000 puzzles); build puzzle representation & validity checker                | Cleaned, labeled dataset (grid, difficulty, solution, empty-cell count) |
| Week 2 | Implement Basic Backtracking (baseline) and instrumentation for metrics (time, nodes, backtracks)                                                                     | Working baseline solver + logging framework                             |
| Week 3 | Implement Backtracking + MRV and Backtracking + Forward Checking                                                                                                      | Two additional solvers, unit-tested                                     |
| Week 4 | Implement MRV + Forward Checking; implement AC-3 constraint propagation as an extended baseline                                                                       | Combined heuristic solver + AC-3 comparison solver                      |
| Week 5 | Run full experiments across all difficulty levels; collect solving time, nodes explored, backtracks, success rate; analyze results and build comparison tables/graphs | Raw results dataset (CSV/JSON), analysis notebook, charts               |
| Week 6 | Build simple demo interface; write final report; prepare code repository and documentation; final review                                                              | Demo system, final technical report, source code                        |

Total duration: 6 weeks, aligned with the AI Lab semester schedule. Compute requirements are modest (all algorithms run on CPU; no GPU/cluster access needed), so the timeline is feasible within standard lab hardware.

## Expected Outcomes and Deliverables

The project is expected to demonstrate how different AI search algorithms and heuristics affect the efficiency of Sudoku solving. The results will identify which approach performs best based on solving time, search nodes, and backtracking. The project will produce:

- Multiple AI-based Sudoku solvers (including an AC-3 constraint-propagation solver)
- Sudoku puzzle dataset with different difficulty levels, with documented source and access method
- Performance comparison of the implemented algorithms
- Tables and graphs showing experimental results
- Sudoku-solving demonstration system
- Final technical report documenting the methodology, results, analysis, and conclusion
- Source code for the algorithms and evaluation system